

Computer Architecture Lab 01 Report

张兆飞

2026-03-11

实验 1 GDB 调试实验

1. 实验目的

了解 gdb 的基本使用方法, 包括设置断点, 查看变量值, 单步执行等功能. 掌握命令行调试程序的技巧, 提高调试效率和能力.

2. 实验步骤

2.1. ex1: 断点的使用

1. 编译程序并启动 gdb:

本次实验环境为个人电脑上的 Ubuntu 24.04 操作系统.

```
jz2024@JZ2024:~/Desktop/Development/Experiments/ComputerArchitecture/Lab01/Projects/gdb_tutorial
jz2024@JZ2024:gdb_tutorial$ ll
total 20K
drwxrwxr-x 2 jz2024 jz2024 4.0K Mar 11 15:49 ./
drwxrwxr-x 3 jz2024 jz2024 4.0K Mar 11 14:40 ../
-rw-rw-r-- 1 jz2024 jz2024 424 Sep 14 2022 ex1.c
-rw-rw-r-- 1 jz2024 jz2024 540 Sep 14 2022 ex2.c
-rw-rw-r-- 1 jz2024 jz2024 862 Sep 14 2022 gdb_example.c
jz2024@JZ2024:gdb_tutorial$ gcc -g ex1.c -o ex1
jz2024@JZ2024:gdb_tutorial$ ll
total 40K
drwxrwxr-x 2 jz2024 jz2024 4.0K Mar 11 15:49 ./
drwxrwxr-x 3 jz2024 jz2024 4.0K Mar 11 14:40 ../
-rwxrwxr-x 1 jz2024 jz2024 18K Mar 11 15:49 ex1*
-rw-rw-r-- 1 jz2024 jz2024 424 Sep 14 2022 ex1.c
-rw-rw-r-- 1 jz2024 jz2024 540 Sep 14 2022 ex2.c
-rw-rw-r-- 1 jz2024 jz2024 862 Sep 14 2022 gdb_example.c
jz2024@JZ2024:gdb_tutorial$ gdb ex1
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ex1...
(gdb)
```

2. 查看程序:

使用 list (l) 命令查看代码.

```

(gdb) 1
1      #include <stdio.h>
2      #include <stdlib.h>
3
4      #define N (123)
5      #define INT_MAX (0x7fffffff)
6
7      int main(int argc, char **argv)
8      {
9          srand(1);
10         int sum = 0;
(gdb)
11         for(int i=1;i<INT_MAX;i++)
12         {
13             sum+=rand()%1000;
14             if(argc>=2 && i==N && sum==atoi(argv[1]))    //atoi:字符串转int
15             {
16                 printf("Test Pass!\n");
17                 return 0;
18             }
19         }
20
(gdb)
21         printf("Test Fail!\n");
22         return 1;
23     }

```

3. 查看变量值:

使用 break (b) 命令设置断点, 并使用 ignore 忽略前 122 次断点触发, 直接跳到第 123 次.

使用 run (r) 运行直至断点处, 再使用 print (p) 命令查看变量值.

```

(gdb) b 14
Breakpoint 1 at 0x1200: file ex1.c, line 14.
(gdb) i b
Num      Type           Disp Enb Address              What
1        breakpoint      keep y   0x0000000000001200 in main at ex1.c:14
(gdb) ignore 1 122
Will ignore next 122 crossings of breakpoint 1.
(gdb) r
Starting program: /home/jz2024/Desktop/Development/Experiments/ComputerArchitecture/Lab01/Projects/gdb_tutorial/ex1

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=1, argv=0x7fffffd8b8) at ex1.c:14
14          if(argc>=2 && i==N && sum==atoi(argv[1]))    //atoi:字符串转int
(gdb) p sum
$1 = 59270

```

4. 实验结果:

输入 59270 作为参数, 程序成功输出 Test Pass!. 本题答案为 59270.

```

jz2024@JZ2024:gdb_tutorial$ ./ex1 59270
Test Pass!

```

2.2. ex2: 打印的使用

1. 编译程序并启动 gdb:

图略.

2. 查看程序:

使用 list (l) 命令查看代码.

```
(gdb) l
1      #include <stdio.h>
2      #include <stdlib.h>
3      #define N (5)
4
5      struct Node
6      {
7          int value;
8          struct Node *next;
9      };
10
(gdb)
11     int main()
12     {
13         srand(1);
14         struct Node *head=NULL,*tail=NULL;
15         for(int i=0; i<N; i++)
16         {
17             struct Node *ptr = (struct Node*)malloc(sizeof(struct Node));
18             ptr->value = rand()%100;
19             ptr->next = NULL;
20             if(head==NULL)
(gdb)
21             {
22                 head=tail=ptr;
23             }
24             else
25             {
26                 tail=tail->next=ptr;
27             }
28             printf("Node %d established.\n",i);
29         }
30         return 0;
(gdb)
31     }
```

3. 查看变量值:

使用 break (b) 命令设置断点. 使用 run (r) 运行直至断点, 再使用 display 将各个变量的值设为显示状态.

```
(gdb) b 28
Breakpoint 1 at 0x1250: file ex2.c, line 28.
(gdb) r
Starting program: /home/jz2024/Desktop/Development/Experiments/ComputerArchitecture/Lab01/Projects/gdb_tutorial/ex2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at ex2.c:28
28      printf("Node %d established.\n",i);
(gdb) display/x head
1: /x head = 0x5555555592a0
(gdb) display/x ptr
2: /x ptr = 0x5555555592a0
(gdb) display/x ptr->value
3: /x ptr->value = 0x53
(gdb) display/x ptr->next
4: /x ptr->next = 0x0
```

使用 continue (c) 继续运行, 观察变量值的变化.

```
(gdb) c
Continuing.
Node 0 established.

Breakpoint 1, main () at ex2.c:28
28      printf("Node %d established.\n",i);
1: /x head = 0x5555555592a0
2: /x ptr = 0x5555555596d0
3: /x ptr->value = 0x56
4: /x ptr->next = 0x0
(gdb)
Continuing.
Node 1 established.

Breakpoint 1, main () at ex2.c:28
28      printf("Node %d established.\n",i);
1: /x head = 0x5555555592a0
2: /x ptr = 0x5555555596f0
3: /x ptr->value = 0x4d
4: /x ptr->next = 0x0
(gdb)
Continuing.
Node 2 established.

Breakpoint 1, main () at ex2.c:28
28      printf("Node %d established.\n",i);
1: /x head = 0x5555555592a0
2: /x ptr = 0x555555559710
3: /x ptr->value = 0xf
4: /x ptr->next = 0x0
(gdb)
Continuing.
Node 3 established.

Breakpoint 1, main () at ex2.c:28
28      printf("Node %d established.\n",i);
1: /x head = 0x5555555592a0
2: /x ptr = 0x555555559730
3: /x ptr->value = 0x5d
4: /x ptr->next = 0x0
(gdb)
Continuing.
Node 4 established.
[Inferior 1 (process 149124) exited normally]
```

4. 实验结果:

head 节点的起始地址为 0x5555555592a0.

节点编号	value (hexadecimal)	next (hexadecimal)
0 (head)	0x53	0x5555555596d0
1	0x56	0x5555555596f0
2	0x4d	0x555555559710
3	0xf	0x555555559730
4 (tail)	0x5d	0x0

3. 实验分析与总结

本次实验主要练习了 gdb 的断点和显示功能, 让我们能够深入理解程序的执行流程和变量的变化情况. 通过设置断点并忽略前几次触发, 我们能够直接跳到感兴趣的代码位置, 提高了调试效率. 显示功能则让我们能够实时观察变量的值, 更加直观地理解程序的行为.

在 ex1 中, 通过设置断点并查看变量值, 我们成功找到了程序的正确输入参数. 在 ex2 中, 通过显示变量值, 我们能够清晰地看到链表节点的结构和内容. 这些技能对于调试复杂程序非常有用, 可以帮助我们快速定位问题并理解程序的运行机制.

4. 实验收获与建议

通过本次实验, 我们不仅掌握了 gdb 的基本使用方法, 还学会了如何高效地调试程序. 这些技能对于我们未来课程的学习和实际项目开发都非常重要.